



UAVLogBook

Docker Install Guide — PC / Dev / Production

Windows · macOS · Linux — Development & Production

Version 1.2 · June 2025

Only need Docker Desktop No PHP, MySQL, or Node installed locally	Dev mode One command make dev Ready in ~3 min	Hot reload included PHP edits live React HMR :3000	Production ready too make up or make prod + SSL
--	--	---	--

Table of Contents

1	System Requirements	3
2	Install Docker Desktop — Windows	4
3	Install Docker Desktop — macOS	5
4	Install Docker Engine — Linux	6
5	Extract and Prepare the Project	7
6	Configure .env	8
7	Start: Development Mode	9
8	Start: Production Mode (no SSL)	11
9	Start: Production Mode with Nginx + SSL	12
10	First Login	13
11	Day-to-Day Dev Workflow	14
12	Useful Commands	15
13	Troubleshooting	16

1 System Requirements

You only need Docker on your PC. PHP, MySQL, Node.js, and all other dependencies run inside containers — nothing else to install.

Minimum PC requirements

Component	Minimum	Recommended
OS	Windows 10, macOS 12, Ubuntu 20.04	Latest version
RAM	4 GB	8 GB+
Disk space	5 GB free	20 GB+ (for video uploads)
CPU	Any 64-bit x86_64 or ARM64	4+ cores
Internet	Required (first run only)	For image downloads
Docker	Docker Desktop 4.x / Engine 24+	Latest

What Docker pulls on first run

Image	Size	Purpose
mysql:8.0	~580 MB	Database server
php:8.2-apache	~480 MB	API + web server (custom build adds ~200 MB)
node:20-alpine	~130 MB	React app builder (exits after build)
nginx:1.25-alpine	~45 MB	Reverse proxy (production profile only)

Total first-run download

Approximately 1.4 GB of Docker images are pulled on the first run. Subsequent starts use cached images — no re-download needed. The PHP image build step (compiling extensions) adds ~5 minutes on first run only.

2 Install Docker Desktop — Windows

Windows 10

Windows 11

WSL 2 required

Docker Desktop on Windows requires WSL 2 (Windows Subsystem for Linux version 2). Windows 10 version 1903 or later supports this. Windows 11 supports it out of the box. The installer will guide you if WSL 2 is not yet enabled.

1

Enable WSL 2 (Windows 10 only — skip on Windows 11)

- Open PowerShell as Administrator (right-click Start → Windows PowerShell (Admin))

```
PS> wsl --install
PS> wsl --set-default-version 2
# Restart Windows when prompted
```

2

Download Docker Desktop

- Go to: <https://www.docker.com/products/docker-desktop/>
- Click 'Download for Windows' — file is named Docker Desktop Installer.exe (~500 MB)

3

Run the installer

- Double-click Docker Desktop Installer.exe
- On the configuration page: ensure 'Use WSL 2 instead of Hyper-V' is checked
- Click OK and wait for installation (~3 minutes)
- Click 'Close and restart' when prompted

4

Start Docker Desktop

- After restart, Docker Desktop launches automatically (whale icon in system tray)
- Wait for it to say 'Docker Desktop is running' (green dot in tray)
- First startup takes 1–2 minutes

5

Verify in PowerShell or Windows Terminal

```
PS> docker --version
Docker version 26.x.x, build xxxxxxx

PS> docker compose version
Docker Compose version v2.x.x

PS> docker run hello-world
Hello from Docker!
```

Windows Terminal recommended

Install Windows Terminal from the Microsoft Store for a better experience. All commands in this guide work in PowerShell, Windows Terminal, or the WSL Ubuntu terminal. The WSL Ubuntu terminal is the most Linux-compatible option.

3 Install Docker Desktop — macOS

macOS Intel

macOS Apple Silicon M1/M2/M3

Apple Silicon fully supported

All Docker images used by UAVLogBook (mysql:8.0, php:8.2-apache, node:20, nginx) are available as multi-arch images and run natively on M1/M2/M3 — no Rosetta emulation needed. Docker Desktop detects your chip automatically.

1 Download Docker Desktop for Mac

- Go to: <https://www.docker.com/products/docker-desktop/>
- Click 'Download for Mac'. Choose the correct version:
- → 'Apple Chip' for M1/M2/M3 MacBook/iMac/Mac Mini
- → 'Intel Chip' for older Intel-based Mac

2 Install Docker Desktop

- Open the downloaded .dmg file
- Drag the Docker icon to the Applications folder
- Open Docker from Applications (or Spotlight: Cmd+Space → 'Docker')
- Click Open when macOS asks for permission
- Enter your Mac password when Docker asks for privileged access

3 Wait for Docker to start

- The Docker whale icon appears in the menu bar (top right)
- Wait until the icon stops animating and shows 'Docker Desktop is running'
- First startup takes 1–2 minutes

4 Verify in Terminal

- Open Terminal (Cmd+Space → 'Terminal') or iTerm2

```
$ docker --version
Docker version 26.x.x, build xxxxxxxx

$ docker compose version
Docker Compose version v2.x.x

$ docker run hello-world
Hello from Docker!
```

4 Install Docker Engine — Linux

Ubuntu 20.04+

Debian 11+

Fedora 38+

On Linux, Docker Engine (not Docker Desktop) is the right choice — it runs natively without a VM layer, giving better performance.

Ubuntu / Debian — Convenience script (fastest method)

Terminal — Ubuntu/Debian

```
# Download and run the official Docker install script
$ curl -fsSL https://get.docker.com | sh

# Add your user to the docker group (so you don't need sudo)
$ sudo usermod -aG docker $USER

# Apply the group change (log out and back in, OR run this)
$ newgrp docker

# Verify
$ docker --version
Docker version 26.x.x, build xxxxxxxx

$ docker compose version
Docker Compose version v2.x.x
```

Fedora / RHEL / CentOS

Terminal — Fedora/RHEL

```
$ sudo dnf install docker-ce docker-ce-cli containerd.io docker-compose-plugin
$ sudo systemctl enable --now docker
$ sudo usermod -aG docker $USER && newgrp docker
```

make command on Linux

Install make if not already present: `sudo apt install make` (Ubuntu/Debian) or `sudo dnf install make` (Fedora). The Makefile shortcuts throughout this guide all use make. Without it, use the full docker compose commands shown alongside each make command.

5 Extract and Prepare the Project

Windows

macOS

Linux

1 Extract the zip file

```
# Windows (PowerShell):
PS> Expand-Archive UAVLogBook_v1.2.zip -DestinationPath C:\Projects\
PS> cd C:\Projects\uavlogbook_v3

# macOS / Linux:
$ unzip UAVLogBook_v1.2.zip
$ cd uavlogbook_v3
```

2 Understand the folder structure

<code>docker-compose.yml</code>	Main stack definition — MySQL, PHP, builder, Nginx
<code>docker-compose.dev.yml</code>	Dev overrides — hot reload, Xdebug, MySQL on :3306
<code>.env.example</code>	Configuration template — COPY this to <code>.env</code>
<code>Makefile</code>	Shortcut commands (make dev, make up, etc.)
<code>.dockerignore</code>	Keeps Docker builds fast
<code>server/</code>	PHP backend source code
<code>web/</code>	React frontend source code
<code>mobile/</code>	React Native mobile app source
<code>docker/php/Dockerfile</code>	PHP 8.2 image definition
<code>docker/nginx/</code>	Nginx config for production SSL
<code>docker/mysql-init/</code>	Database schema — auto-imported on first start
<code>docker/synology/</code>	Synology DS412+ specific files — ignore for PC

6 Configure .env

Windows

macOS

Linux

All configuration lives in one `.env` file. You must create it from the template before starting any containers.

1 Copy the template

```
# macOS / Linux:
$ cp .env.example .env

# Windows (PowerShell):
PS> Copy-Item .env.example .env
```

2 Edit .env — set these 3 required values

```
# Open in your editor:
$ nano .env                # macOS/Linux terminal
$ code .env                # VS Code (any platform)
PS> notepad .env           # Windows Notepad

# The ONLY values you MUST change for dev:
MYSQL_ROOT_PASSWORD=any_strong_password_here
MYSQL_PASSWORD=any_different_strong_password
JWT_SECRET=paste_64_random_chars_here

# Generate JWT_SECRET with this command:
$ openssl rand -hex 32
# Copy the output and paste it as the JWT_SECRET value

# Windows PowerShell alternative (if openssl not available):
PS> [System.Web.Security.Membership]::GeneratePassword(64,8)
```

Complete .env reference — all settings explained

MYSQL_ROOT_PASSWORD	REQUIRED	MySQL root password. Any strong string. Not used by the app directly.
MYSQL_DATABASE	<code>uavlogbook</code>	Database name. Leave as-is unless you have a reason to change it.
MYSQL_USER	<code>uavlogbook_user</code>	App DB username. Leave as-is.
MYSQL_PASSWORD	REQUIRED	App DB password. Must differ from root password.

JWT_SECRET	REQUIRED	Token signing secret. Generate with: openssl rand -hex 32
ANTHROPIC_API_KEY	(blank)	Optional. Claude AI key for smart log format detection. Leave blank to disable.
VITE_API_URL	/api/v1	API path for the React app. Leave as /api/v1 — Docker proxy handles routing.
ALLOWED_ORIGINS	http://localhost, ...	CORS origins. For dev, the default covers localhost:3000 and :8080.
NGINX_SERVER_NAME	localhost	Your domain name. Only used in production Nginx profile.
PHP_MEMORY_LIMIT	256M	PHP memory. 256M is fine for dev. Increase for very large log files.
PHP_UPLOAD_MAX_FILESIZE	8192M	Max upload size. 8GB covers the largest FPV video files.
APP_PORT	8080	Host port for the app. Change if 8080 is already in use on your PC.

7 Start: Development Mode

Windows

macOS

Linux

Development mode gives you hot reload for both PHP and the React app. Edit any file and see changes instantly in the browser — no rebuild needed.

1

Open a terminal in the project folder

```
# macOS / Linux:
$ cd /path/to/uavlogbook_v3

# Windows (PowerShell or Windows Terminal):
PS> cd C:\Projects\uavlogbook_v3

# Windows tip: in File Explorer, Shift+Right-click the folder
# → 'Open PowerShell window here' or 'Open in Terminal'
```

2

Start the dev stack

```
# With make (macOS/Linux):
$ make dev

# Without make (all platforms):
$ docker compose -f docker-compose.yml -f docker-compose.dev.yml up --build

# Windows PowerShell:
PS> docker compose -f docker-compose.yml -f docker-compose.dev.yml up --build
```

3

Watch the startup sequence

The logs stream to your terminal. Here is what you should see:

Terminal output — what you should see

```

uavlogbook_db      | Starting MySQL 8.0...
uavlogbook_db      | ready for connections.          ← MySQL ready

uavlogbook_app     | Building PHP image...
uavlogbook_app     | Successfully built xxxxxxxx    ← Image built (first run only)
uavlogbook_app     | AH00094: Command line: ...     ← Apache started

uavlogbook_webdev  | > vite --host 0.0.0.0
uavlogbook_webdev  | VITE v5.x ready in 1234 ms     ← React dev server ready
uavlogbook_webdev  | → Local: http://localhost:3000/

```

4**Open the app**

```

React app (Vite HMR):      http://localhost:3000
API direct access:        http://localhost:8080/api/v1
MySQL (DB tools):         localhost:3306

# Username/password for MySQL tools (e.g. TablePlus, DBeaver):
Host:      localhost
Port:      3306
User:      uavlogbook_user    (MYSQL_USER in .env)
Password:  your_db_password  (MYSQL_PASSWORD in .env)
Database:  uavlogbook         (MYSQL_DATABASE in .env)

```

First run timing

The very first docker compose up takes 5–15 minutes: ~1.4 GB of images are downloaded, then the PHP image is built (compiles extensions). Every subsequent start takes under 30 seconds — images are cached locally.

What 'development mode' gives you

PHP hot-edit — Edit any .php file in server/ and refresh your browser — changes apply instantly. The server/ directory is bind-mounted read-write into the container.

React HMR :3000 — Edit any .jsx/.js/.css file in web/src/ and the browser updates in milliseconds without losing state. Powered by Vite's Hot Module Replacement.

Xdebug on :9003 — Step-debug PHP with VS Code or PhpStorm. The PHP container listens for debugger connections automatically.

MySQL on :3306 — Connect any database GUI (TablePlus, DBeaver, MySQL Workbench) to localhost:3306 to browse tables, run queries, and inspect data.

Error display — PHP errors are shown directly in the browser response body (PHP_DISPLAY_ERRORS=On). In production this is Off.

5

Stop the dev stack

```
# In the terminal where it's running:  
Press Ctrl+C  
  
# Or from another terminal:  
$ make down  
$ docker compose down  
  
# Data in volumes (database, uploads) is KEPT after stopping  
# To also delete all data (full reset):  
$ make clean    # asks for confirmation first
```

8 Start: Production Mode (no SSL)

Use this when you want the full production stack (React app pre-built, Opcache enabled, no Xdebug) but don't need Nginx SSL — e.g. running on a local server, behind an existing proxy, or for testing.

```
$ make up
# or:
$ docker compose up -d

# What happens on first start:
# 1. MySQL starts and imports schema automatically
# 2. Builder runs npm ci && npm run build → React app compiled
# 3. App waits for MySQL health check, then Apache starts

# Check status:
$ make ps
$ docker compose ps

# App is at:
http://localhost:8080

# Follow logs:
$ make logs
$ docker compose logs -f
```

Differences from dev mode

React app	Pre-built at startup by builder container (not Vite HMR)
PHP errors	Hidden from browser — check docker compose logs app
Opcache	Enabled — PHP runs faster, but you must restart app after edits
MySQL port	NOT exposed on host (internal only — more secure)
Xdebug	Not installed in production image
React port :3000	Not available — app served from :8080 directly

9 Start: Production Mode with Nginx + SSL

Use this for a real internet-facing server. Nginx handles SSL termination, HTTP→HTTPS redirect, and rate limiting.

DNS must point to this server first

Before running certbot, your domain's A record must point to this machine's public IP address. DNS propagation can take up to 60 minutes. Check with: `nslookup yourdomain.com` — it should return your server IP.

```
# 1. Edit .env — set your domain
NGINX_SERVER_NAME=yourdomain.com
ALLOWED_ORIGINS=https://yourdomain.com

# 2. Start full stack with Nginx
$ make prod
# or:
$ docker compose --profile production up -d

# 3. Get free SSL certificate (Let's Encrypt)
$ make ssl D=yourdomain.com E=admin@yourdomain.com
# or:
$ bash docker/nginx/certbot.sh yourdomain.com admin@yourdomain.com

# 4. Verify
https://yourdomain.com           # web app
https://yourdomain.com/api/v1    # API
http://yourdomain.com            # redirects to HTTPS automatically
```

10 First Login

1 Open the app in your browser

Development: `http://localhost:3000`

Production: `http://localhost:8080`

You should see the UAVLogBook login screen.

If you see a blank page or error – see Section 13 (Troubleshooting).

2 Quick API health check

Before logging in, confirm the API is responding:

macOS/Linux:

```
$ curl -s -X POST http://localhost:8080/api/v1/auth/login \  
-H 'Content-Type: application/json' \  
-d '{"email":"x","password":"x"}'
```

Expected: `{"error":"Invalid credentials"}`

This confirms Apache, PHP, and MySQL are all working.

Windows PowerShell:

```
PS> Invoke-WebRequest -Uri http://localhost:8080/api/v1/auth/login \  
-Method POST -ContentType 'application/json' \  
-Body '{"email":"x","password":"x"}'
```

3 Register your account

- Click Register on the login screen
- Enter your email address
- Choose a password (minimum 8 characters)
- Enter a display name
- Click Create Account — you are now logged in
- The first account registered becomes admin

4 Import your first flight log

- Click Import Log in the left sidebar
- Drag and drop any supported log file onto the drop zone
- Supported: `.bin` `.tlog` `.ulg` `.skylog` `.csv` `.gpx` `.kml` `.bbl`
- The AI engine detects the format automatically
- After import, click the flight to open the analysis dashboard

11 Day-to-Day Dev Workflow

Editing PHP (server/)

In dev mode, the `server/` directory is bind-mounted into the container. Edit any `.php` file and refresh the browser — no restart needed.

```
# Edit PHP files directly:
$ code server/api/index.php
$ code server/parsers/skyline_parser.php

# Changes are live immediately — just refresh your browser
# PHP errors appear in the browser (dev mode) or in logs:
$ make logs-app
```

Editing React (web/src/)

Vite HMR updates the browser within milliseconds of saving a file. No page refresh needed — React state is preserved.

```
# Edit React files:
$ code web/src/App.jsx
$ code web/src/VideoSync.jsx

# Browser at http://localhost:3000 updates instantly
# No build step, no restart
```

Switching between dev and production

```
# Stop dev stack:
$ make down

# Start production stack:
$ make up

# Switch back to dev:
$ make down
$ make dev

# Database and upload volumes persist across switches
```

Running database queries

```
# Open MySQL shell directly:
$ make db-shell

# Or connect with a GUI tool to localhost:3306 (dev mode only)
# TablePlus, DBeaver, MySQL Workbench, HeidiSQL

# Useful queries:
mysql> SELECT id, email, created_at FROM users;
mysql> SELECT id, parse_status, log_format, original_filename
        FROM flights ORDER BY id DESC LIMIT 10;
mysql> SELECT COUNT(*) FROM telemetry_gps WHERE flight_id=1;
```

Xdebug step-debugging (VS Code)

```
# The PHP container already listens on port 9003 in dev mode.
# Add this to .vscode/launch.json in the project root:

{
  "version": "0.2.0",
  "configurations": [{
    "name": "UAVLogBook Docker PHP",
    "type": "php",
    "request": "launch",
    "port": 9003,
    "pathMappings": {
      "/var/www/html/api": "${workspaceFolder}/server"
    }
  }]
}

# In VS Code: Run → Start Debugging (F5)
# Set breakpoints in any .php file — they will be hit on API calls
```

12 Useful Commands

All make commands run from the project root (uavlogbook_v3/). The full docker compose equivalent is shown for Windows/no-make users.

Starting & stopping

make dev	Start dev stack (foreground — Ctrl+C to stop)
■ docker compose -f docker-compose.yml -f docker-compose.dev.yml up --build	
make dev-d	Start dev stack in background
■ docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d --build	
make up	Start production stack (no Nginx) in background
■ docker compose up -d	
make prod	Start full production stack with Nginx
■ docker compose --profile production up -d	
make down	Stop and remove all containers (data kept)
■ docker compose --profile production down	

Logs & monitoring

make logs	Tail logs from all containers
■ docker compose logs -f	
make logs-app	PHP/Apache logs only
■ docker compose logs -f app	
make logs-db	MySQL logs only
■ docker compose logs -f db	
make ps	Show container status
■ docker compose ps	
docker stats	Real-time CPU/RAM usage
■ docker stats	

Shells & access

make shell	Bash shell inside PHP container
■ docker compose exec app bash	
make db-shell	MySQL shell in database container
■ docker compose exec db mysql...	

Maintenance

make rebuild-web	Force rebuild React app (after web/src changes in production)
■ Remove web_dist volume + re-run builder	
make rebuild-app	Rebuild PHP image (after Dockerfile changes)
■ docker compose build --no-cache app	
make backup	Backup database to local backups/ folder
■ mysqldump → backups/uavlogbook_TIMESTAMP.sql	

```
make restore  
F=backups/file.sql
```

Restore database from a backup file

■ `mysql < file`

```
make ssl D=domain E=email
```

Get or renew Let's Encrypt SSL certificate

■ `bash docker/nginx/certbot.sh`

```
make clean
```

DESTRUCTIVE: removes all containers + volumes (asks for confirmation)

■ `docker compose down -v --remove-orphans`

13 Troubleshooting

'docker compose' not found (Windows / older systems)

Docker Compose v2 ships as a plugin — it is 'docker compose' (space), not 'docker-compose'. If you get 'unknown command', update Docker Desktop to the latest version.

```
$ docker --version          # must be 24.0+
$ docker compose version    # must show v2.x.x

# If docker compose is missing, update Docker Desktop:
# Windows/macOS: Help → Check for Updates in Docker Desktop
# Linux: sudo apt-get update && sudo apt-get install docker-compose-plugin
```

Port 8080 already in use

Another application is using port 8080. Either stop it or change APP_PORT in .env.

```
# Find what is using port 8080:
$ lsof -i :8080                # macOS/Linux
PS> netstat -ano | findstr :8080 # Windows PowerShell

# Change the port in .env:
APP_PORT=9090

# Then restart:
$ make down && make up
```

App container keeps restarting

Usually a wrong password in .env, or MySQL is not yet healthy. Check logs.

```
$ docker compose logs app      # see PHP error
$ docker compose logs db       # see MySQL error

# Common causes:
# - .env not created (cp .env.example .env)
# - MYSQL_PASSWORD mismatch between db and app sections
# - MySQL still initialising — wait 60s and check again
```

Blank white page in browser

In production mode the React app was not built yet, or the builder failed. In dev mode, Vite may still be starting up.

```
# Production: check builder logs
$ docker compose logs builder

# Force a rebuild:
$ make rebuild-web

# Dev: check Vite is running
$ docker compose logs webdev
# Vite takes ~10s to start — wait and retry
```

MySQL health check failing — app won't start

MySQL is still initialising. On first run it imports the schema which takes 30–60 seconds.

```
$ docker compose logs -f db
# Wait for: ready for connections

# If it never becomes healthy, check the password matches:
$ docker compose logs db | grep -i error

# Reset database (WARNING: deletes all data):
$ docker compose down
$ docker volume rm uavlogbook_db_data
$ docker compose up -d
```

File upload fails or times out (large video files)

PHP limits or Nginx client_max_body_size may be too low. Check the .env values and restart the app container.

```
# Check .env limits:
PHP_UPLOAD_MAX_FILESIZE=8192M
PHP_POST_MAX_SIZE=8192M
PHP_MAX_EXECUTION_TIME=600

# Restart app to apply:
$ docker compose restart app
```

'Permission denied' running docker on Linux

Your user is not in the docker group yet.

```
$ sudo usermod -aG docker $USER
$ newgrp docker          # apply immediately without logout

# Or prefix with sudo temporarily:
$ sudo docker compose up -d
```

Docker Desktop very slow on Windows

Ensure your project folder is inside the WSL 2 filesystem for best performance. Windows filesystem (C:\) mounted into WSL is much slower than native WSL paths.

```
# Clone/extract the project INTO WSL filesystem:
# Open WSL terminal and use:
$ cd ~
$ cp /mnt/c/Downloads/UAVLogBook_v1.2.zip .
$ unzip UAVLogBook_v1.2.zip
$ cd uavlogbook_v3
$ make dev

# VS Code can open WSL folders with the WSL extension:
$ code .
```